

Academic integrity declaration

By submitting work for assessment, I hereby declare that I understand the University’s policy on academic integrity and the rules of the assignment as set by the subject coordinator regarding the use of artificial intelligence software, including editing and translation tools.

I declare that the submission is original and solely my work, and that I have not been assisted by another person (collusion) apart from where the submitted work is for a designated collaborative task, in which case the individual contributions are indicated. All sources and tools used have been appropriately cited or acknowledged, and any copied or modified code has been declared using the methods indicated by my subject coordinator.

Assignment 1a - Concurrency in Java

Due: Thu Apr 2, 2026 23:59

11.25/12.5 Points

Attempt 1



Review Feedback
SUBMITTED: Apr 2, 2026 14:27

Attempt 1 Score:
11.25/12.5

Add Comment

Anonymous Marking: **no**

Unlimited Attempts Allowed

Details

Weight: 12.5%

Due: 11:59pm, Monday 30 March 2026

Submission: Submit your assignment by uploading a file on this screen and following the prompts.

Questions: Please use the [Ed Discussion Forum \(https://canvas.lms.unimelb.edu.au/courses/237797/external_tools/3724?display=borderless\)](https://canvas.lms.unimelb.edu.au/courses/237797/external_tools/3724?display=borderless) for questions about the assignment - note: please do not post code there!

Extensions: Please see the [FEIT Extensions and Special consideration \(https://canvas.lms.unimelb.edu.au/courses/237797/pages/feit-extensions-and-special-consideration\)](https://canvas.lms.unimelb.edu.au/courses/237797/pages/feit-extensions-and-special-consideration) page.

Academic integrity: Please see the [Academic Integrity \(https://canvas.lms.unimelb.edu.au/courses/237797/pages/academic-integrity\)](https://canvas.lms.unimelb.edu.au/courses/237797/pages/academic-integrity) page.

Scaffold code: <https://canvas.lms.unimelb.edu.au/courses/237797/files/26622410?wrap=1> **SWEN90004 2026 Asg1a Scaffold.zip**
<https://canvas.lms.unimelb.edu.au/courses/237797/files/26622410?wrap=1> ↓ https://canvas.lms.unimelb.edu.au/courses/237797/files/26622410/download?download_frd=1

Sample solution: [SWEN90004 2026 Asg 1a Solution.zip \(https://canvas.lms.unimelb.edu.au/courses/237797/files/26869566?wrap=1\)](https://canvas.lms.unimelb.edu.au/courses/237797/files/26869566?wrap=1) ↓
https://canvas.lms.unimelb.edu.au/courses/237797/files/26869566/download?download_frd=1

Notes on sample solution: You will see that I have modelled the Aircraft and Operator classes by implementing the Runnable interface, rather than by extending the Thread class; however, either of these is fine from a marking perspective (all other things being equal). Also, I have abstracted Runways and Gates such that only the number of each is tracked by the Airport class. An alternative solution that implements Runway and Gate classes is also possible and acceptable from a marking perspective.

Introduction

There are two related parts to Assignment 1. This first part of the assignment (1a, the current part) deals with programming concurrency in Java using threads and monitors, and is worth 12.5% of your final mark. The second part of the assignment (1b, which will be released in a few weeks) deals with modelling concurrency in FSP and is also worth 12.5% of your final mark. You should complete both parts of Assignment 1 individually.

Your task in this part of the assignment is to implement a concurrent simulation of an Airport proposed to be built in Semaphora, the capital of Concurrancia. The purpose of this simulation is to allow the Airport's management to be confident that it will operate smoothly under different demand conditions.

The system to simulate

Passenger **aircraft** arrive at and depart from the **airport** using a fixed number of **runways** (R) and **gates** (G). Runways are required for arriving aircraft to **land** and for departing aircraft to **takeoff**. Each runway can only be occupied by a single aircraft at a time. Gates are required for aircraft to park after they have landed. Each gate can only be occupied by a single aircraft at a time. Landed aircraft remain at their gate for **turnaround** before taking off again, during which time passengers disembark and embark and the aircraft is serviced and refuelled.

In order to land, an arriving aircraft must obtain both a runway and a gate. Aircraft may not occupy a runway while waiting for a gate. In order to takeoff, a departing aircraft (located at a gate) must obtain a runway.

The airport has a policy of **arrivals priority**: during normal operations, arrivals should have priority over departures if both are able to proceed. The purpose of this policy is to ensure that arriving aircraft do not run low on fuel before having a chance to land.

Semaphora airport is allowed to operate only during daylight hours and must close at night. When the airport is **open**, both landing and takeoff are permitted. At dusk, an **operator** begins to **close** the airport. From this time, no new aircraft may land, but aircraft currently at the airport may takeoff. Once no aircraft remain, the operator closes the airport fully and no further landings or takeoffs are permitted until they reopen the airport the following morning.

When implementing your simulation, you should make the following **assumptions**:

- Aircraft do not crash or experience other emergencies.
- Runways and gates are always operational.
- Landing, takeoff, and turnaround are finite duration operations.
- No aircraft remain at the airport overnight (while it is closed).

When implementing your simulation, you do **not** need to:

- Guarantee fairness.
- Model passengers.
- Model fuel limits.
- Model taxiing between the runway and the gate.
- Model the behaviour of aircraft prior to requesting landing or after taking off.

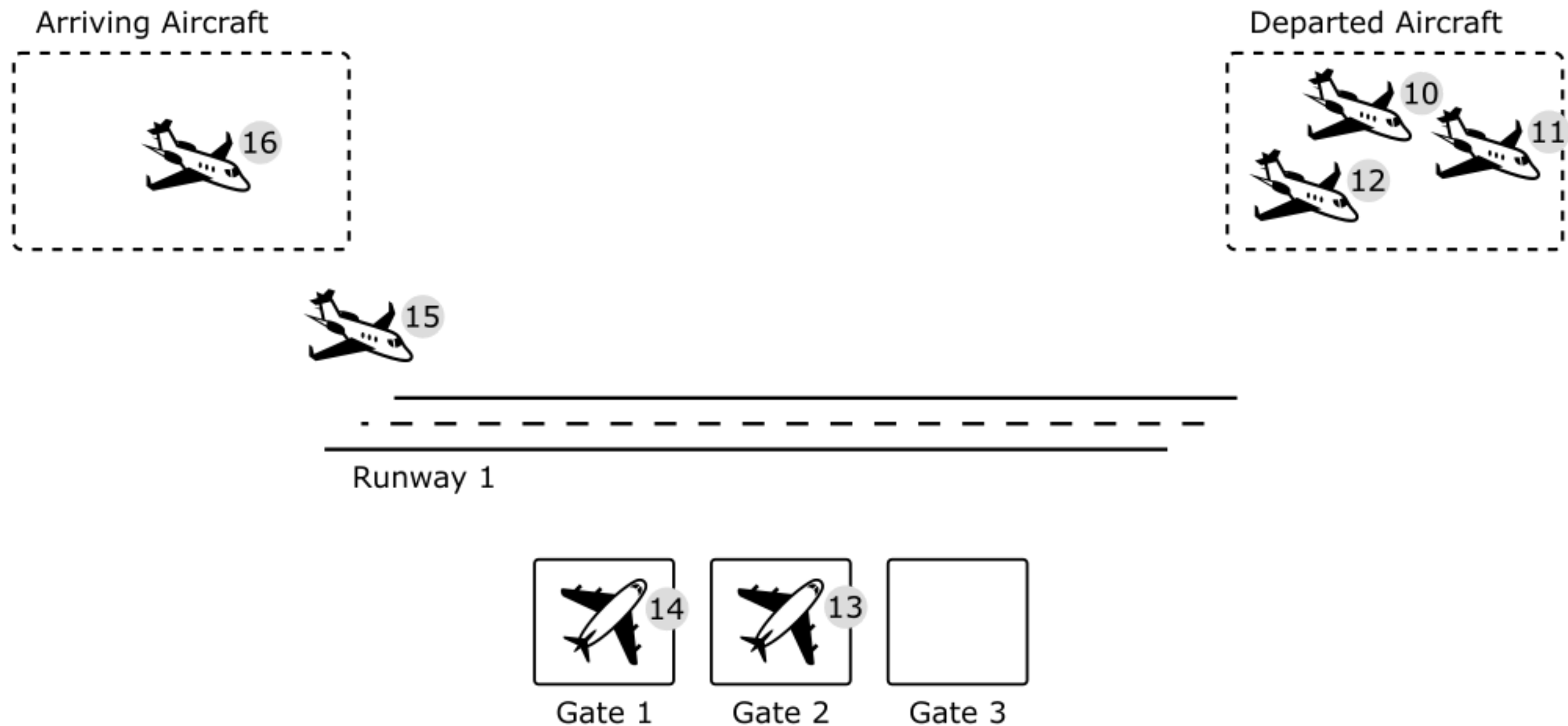


Figure: A schematic diagram of Semaphora airport. The diagram shows: 1. One aircraft (16) that is currently waiting to land. 2. One aircraft (15) that has obtained both a runway (Runway 1) and a vacant gate (Gate 3) and is now landing. 3. Two aircraft (13 and 14) that have landed at the airport and are now in turnaround or waiting to obtain a runway so that they can takeoff. 4. Three aircraft (10, 11 and 12) that have already taken off and departed - these are no longer represented in the simulation but shown here for illustration. This state of the system shown in this figure corresponds to the end of the trace of events below.

Assignment Tasks

Task 1: Implementation

Your first task is to implement a simulation of the Semaphora Airport system described above. Your implementation should use Java's low-level concurrency primitives, as described in class. You should **not** use higher-level concurrency classes provided by `java.util.concurrent` (e.g., `ReentrantLock`, `BlockingQueue`, `Semaphore`, etc).

[FAQ: why can't I use these higher-level classes ? Answer: Because those higher-level classes solve the hard problems for you. In this subject we want you to understand *how* those problems are solved!]

Your implementation should be suitably parameterised such that the timing assumptions can be varied, and the number of runways and gates can be varied. You should use your simulator to explore the behaviour of the system to identify any potential problems with its operation.

The simulation should produce a trace of events matching the (partial) trace below, generated after the system has been running for some time. As the behaviour of the simulation will be non-deterministic, the output from your code won't match this example exactly, but the format of the trace messages should match.

At the end of the trace, the state of the system corresponds to the figure above.

```
:
[Aircraft_10] Requesting takeoff
[Aircraft_10] Taking off
[Aircraft_10] Departed
[Aircraft_12] Requesting landing
[Aircraft_12] Landing
[Aircraft_12] At gate
[Operator] Operator begins closing airport.
[Operator] Waiting for all aircraft to depart...
[Aircraft_12] Requesting takeoff
[Aircraft_12] Taking off
[Aircraft_12] Departed
[Operator] Airport is now fully closed.
[Aircraft_13] Requesting landing
[Aircraft_13] Cannot land, airport closed
[Aircraft_14] Requesting landing
[Aircraft_14] Cannot land, airport closed
[Aircraft_13] Requesting landing
[Aircraft_13] Cannot land, airport closed
[Operator] Airport is open. Normal operation.
[Aircraft_14] Requesting landing
[Aircraft_14] Landing
[Aircraft_13] Requesting landing
[Aircraft_14] At gate
[Aircraft_13] Landing
[Aircraft_13] At gate
[Aircraft_15] Requesting landing
[Aircraft_15] Landing
[Aircraft_16] Requesting landing
:
```

Scaffold code and suggested approach

Scaffold code is provided to get you started, including the following classes:

Main: The overall driver of the simulation. Note that this won't compile until you have defined additional classes: an Airport, Aircraft, and an Operator responsible for opening and closing the airport. You should not need to modify this file.

Params: Gathers various system-wide parameters, including number of runways and gates, timing assumptions, and functions for sleeping and for logging messages to produce a trace. You will want to experiment with the system parameters and timing assumptions

While this system is relatively simple, correctly implementing the interactions between various components can be tricky. I suggest approaching this task in two stages:

- First, assume that the airport is **always** open (ie, the operator never closes it) and focus on implementing the interactions between the Aircraft and Airport.
- Once you are confident that this simplified version of the system is working correctly, add in the Operator to open and close the airport, adjusting the Aircraft behaviour accordingly.

Task 2: Reflection

Your second task is to write a reflection (strictly between 400 and 500 words) that addresses the following:

- Do you believe your implementation is correct? How confident are you in this assessment? What evidence supports your claim?
- Identify one safety property of your system and explain how your code enforces it.
- Discuss whether your implementation guarantees liveness and justify your reasoning.
- Describe one concurrency issue you encountered when developing your implementation.
- Explain how you detected and resolved this issue.

You should refer to relevant parts of your code to support your reflection, either by including short snippets of code (which do **not** count toward your word limit) or by referring to source file and line numbers.

Procedure and assessment

- The project must be completed individually and all submitted work must be your own work.
- You should submit a single zip file with the following structure:
 - *<student number>_<name>.zip*
 - *src*
 - *Main.java*
 - *Params.java*
 - *etc (all other .java source files)*
 - *reflection.txt*
- All source files and your reflection must contain your student number and name at the top of each file.
- 1 mark per calendar day (ie, including Saturday and Sunday) will be deducted for late submissions.
- Marks will be awarded according to the rubric below.

Note: Please make sure that:

- you have acknowledged and referenced any external sources (including any GenAI) you use.
- your code compiles without further dependencies.
- you include your full source code (ie, **not** compiled bytecode .javac files).
- your reflection is in plain text.
- all files contain your name and student number.

▼ **View Rubric**

Select Marker

Colton Carner (TA) ▼

SWEN90004-A1a-2025		
Criteria	Ratings	Points

Criteria	Ratings				Points
Understanding view longer description	Outstanding Demonstrates a very high level of understanding of concurrency. 5 to >4 pts	Proficient Demonstrates a good level of understanding of concurrency, with areas for improvement. 4 to >3 pts	Developing Demonstrates a moderate understanding of concurrency with opportunity for improvement. 3 to >2 pts	Not yet proficient Demonstrates substantial gaps in understanding of concurrency with substantial opportunity for improvement across multiple areas. 2 to >0 pts	5/5 pts
Comment - threads + monitors used appropriately - good use of synchronized + wait/while					
Correctness view longer description	Matching or mostly consistent 2 to >1 pts	Substantial inconsistency 1 to >0 pts	Code could not be executed 0 pts	2/2 pts	
Comment - trace looks good - implementation of plane priority looks good.					
Code quality view longer description	Very good Highly readable code. 1.5 to >1 pts	Acceptable Moderately readable code. Some opportunities for improvement. 1 to >0.5 pts	Insufficient Low level of readability. Substantial opportunity for improvement. 0.5 to >0 pts	1.25/1.5 pts	
Comment - in 'requestLanding' method it has a while/wait loop for when the runway/gate is not available, but doesn't while/wait when the airport itself is closed (just bails), therefore the calling code ALSO has to have a while loop around the requestLanding func call. Might be cleaner to have all the waiting/sleeping in one place (can use Object.wait(timeoutMs) in place of sleep so it gives up the monitor lock). Would also result in the requestLanding function being a bit cleaner, not having to check state multiple times. - good commenting and structure					
Reflection view longer description	Outstanding Demonstrates deep insight and reflection on learning. 4 to >3 pts	Acceptable Some evidence of insight and reflection on learning. 3 to >1 pts	Insufficient Little evidence of insight and reflection on learning. 1 to >0 pts	3/4 pts	
Comment - Good discussion of confidence in correctness. Perhaps a small amount more consideration of what specific parameter combinations you tested would be ideal (eg, choosing one with many runways but few gates would test to ensure that deadlocks caused by planes landing and locking runways but not gates was not present; etc), and also mentioning how long you ran the tests for each time (30seconds? 2 hours?) - Good consideration of liveness for takeoff, but there's also a potential liveness issue for landing planes to consider too!					